

7. 步骤六：注册知识文档与投递语义任务

正式 root_uri 已经确定后，ResourceService 的 post_finalize_hook 会注册知识文档记录。随后 ResourceProcessor 根据 build_index 和 summarize 判断是否需要语义处理。默认 build_index=true，因此一般都会投递 SemanticMsg。

```
ResourceProcessor after move/finalize
should_summarize = summarize or build_index
processing_requested = should_summarize

post_finalize_hook(result):
    register knowledge document
    processing_status = processing if should_summarize else ready
    result.document_id = document_id

Summarizer.summarize(
    resource_uris=[root_uri],
    temp_uris=[temp_uri_for_summarize],
    skip_vectorization=not build_index,
    lifecycle_lock_handle_id=handle_id,
    document_id=document_id
)

SemanticQueue.enqueue(SemanticMsg)
```

SemanticMsg 字段	值来源	作用
uri	temp_uri 或 root_uri	SemanticProcessor 实际读取和处理的目录。
target_uri	root_uri, 当 uri 与 root_uri 不同	增量/同步时用于把临时树同步到正式树。
context_type	由 root_uri 推导，一般为 resource	决定向量记录和上下文类型。
account_id/user_id/agent_id/role	RequestContext	用于权限、owner_space 和索引过滤。
skip_vectorization	not build_index	允许只生成摘要，不写向量索引。
lifecycle_lock_handle_id	ResourceProcessor 获取的 SUBTREE 锁	保护语义处理和增量同步期间的目录一致性。
is_code_repo	source_format == repository	让 SemanticDag 按代码仓库策略处理。
document_id	KnowledgeDocumentRegistry	处理完成后更新 ready/failed 状态。

8. 步骤七：SemanticProcessor 生成 L0/L1 并向量化

SemanticProcessor 从 SemanticQueue 取出 SemanticMsg。如果是 resource 类型，它会创建 SemanticDagExecutor，从根目录开始调度目录节点。DAG 的核心策略是自底向上：先处理文件和子目录，再生成父目录 overview/abstract。

```
SemanticProcessor.on_dequeue(msg)
-> ctx_from_semantic_msg(msg)
-> if msg.target_uri exists and msg.uri != msg.target_uri:
    incremental_update = true
```